

โครงการ: การแข่งขันพัฒนาระบบควบคุมรถยนต์ด้วย AI & Prompt Engineering

ชื่อ-นามสกุล: นายตะวัน ตะเพียนทอง

รหัสนักศึกษา: 65120501058

1. บทนำ (Introduction)

การพัฒนาระบบควบคุมรถยนต์อัตโนมัติเป็นหนึ่งในความท้าทายที่สำคัญในวงการปัญญาประดิษฐ์ ซึ่งต้องอาศัยการผสมผสานความรู้ทั้งด้าน Computer Vision, Machine Learning, และ Robotics โครงการ "การแข่งขันพัฒนาระบบควบคุมรถยนต์ด้วย AI & Prompt Engineering (ปีการศึกษา 2568)" นี้มีวัตถุประสงค์ให้นักศึกษาได้ประยุกต์ใช้ความรู้ดังกล่าวเพื่อพัฒนาโปรแกรมควบคุมรถยนต์จำลองในสนามแข่ง

ในโปรเจกต์นี้ ผู้พัฒนาได้มุ่งเน้นไปที่การสร้างระบบควบคุมรถยนต์ที่สามารถขับเคลื่อนตามเส้นกลางถนนได้อย่างอัตโนมัติ โดยใช้ข้อมูลภาพจากกล้องที่ติดตั้งบนรถยนต์เป็น Input หลัก แนวคิดสำคัญคือการพัฒนา Regression Model บนสถาปัตยกรรม ResNet18 ที่ผ่านการ Pre-trained เพื่อทำนายพิกัดเป้าหมาย (x,y) บนเส้นกลางถนน ค่าพิกัดที่ได้นี้จะถูกนำไปใช้ในการคำนวณมุมเลี้ยว (Steering Angle) และความเร็วของรถยนต์ เพื่อให้รถสามารถเคลื่อนที่ตามเส้นทางที่กำหนดในสนามแข่งของโปรแกรม Unity ได้อย่างเหมาะสม นอกจากนี้ เพื่อให้การควบคุมมีความนิ่งและสมจริงยิ่งขึ้น ได้มีการเพิ่ม Logic การตรวจสอบ "ความมั่นใจ" ของ AI ในการตัดสินใจเลี้ยว

กระบวนการพัฒนาทั้งหมด ตั้งแต่การเตรียมข้อมูล, การออกแบบและฝึกฝนโมเดล, จนถึงการปรับปรุงโค้ดควบคุมรถ ได้มีการประยุกต์ใช้เครื่องมือ Generative AI (Gemini) ผ่านการเขียน Prompt อย่างมีประสิทธิภาพ เพื่อเป็นผู้ช่วยในการระดมสมอง, แก้ไขปัญหาโค้ด, และสร้างโครงร่างเนื้อหา, ซึ่งเป็นไปตามวัตถุประสงค์ของการแข่งขันที่ต้องการให้นักศึกษาได้เรียนรู้และทดลองใช้เครื่องมือ AI สมัยใหม่

2. การเตรียมข้อมูล

2.1 แหล่งข้อมูล

ข้อมูลภาพที่ใช้ในการฝึกฝนโมเดลเป็นข้อมูลที่เคยได้รับในการฝึกปฏิบัติในชั้นเรียน (Lec09-การฝึกปฏิบัติในชั่วโมง.pdf)

ประเภทแหล่งภาพ:

☐ Live (Webcam/URL)

☒ ไฟล์วิดีโอ

เลือกจากโฟลเดอร์: Recording 2025-05-07 144050.mp4

📁 หรือ Upload ไฟล์...

▶ เริ่ม / Load แหล่งภาพ... ■ จบการทำงาน

เลือกประเภทและแหล่งภาพ แล้วกด 'เริ่ม / Load'

Category: forward 📷 ถ่ายรูปเก็บข้อมูล (C)

👉 เลือก ROI 🗑 ล้าง ROI

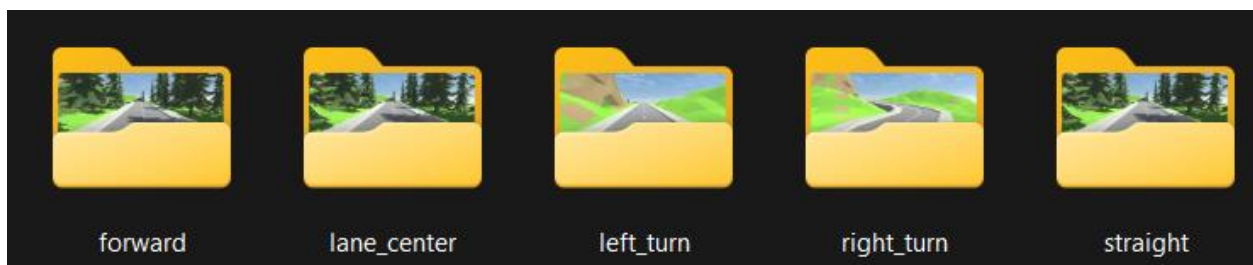
ROI: ไม่ได้เลือก

ชื่อ Category ใหม่: เช่น forward, lane_center + เพิ่ม Category

(รูปที่ 1: การเก็บ Dataset สำหรับการเทรนโมเดล)

2.2 ลักษณะของข้อมูลเริ่มต้น

Dataset ดั้งเดิมประกอบด้วยรูปภาพถนน ซึ่งถูกจัดเก็บแยกตามโฟลเดอร์ย่อยที่แสดงลักษณะการขับขี่ต่างๆ ได้แก่ forward, lane_center, left_turn, right_turn, และ straight



(รูปที่ 2: ตัวอย่าง Dataset สำหรับการเทรนโมเดล)

2.3 การประมวลผลข้อมูลเบื้องต้น (Preprocessing) และการสร้าง Label สำหรับ Regression

เนื่องจากเป้าหมายคือการพัฒนา Regression Model เพื่อทำนายพิกัด (x,y) ของเส้นกลางถนน จึงจำเป็นต้องสร้าง Label ที่แม่นยำสำหรับแต่ละภาพใน Dataset ขั้นตอนนี้ดำเนินการโดยใช้สคริปต์ create_regression_labels.py ที่พัฒนาขึ้นเอง (โดยได้รับความช่วยเหลือจาก Generative AI (Gemini) ในการร่างโครงสร้างเบื้องต้น) สคริปต์นี้จะแสดงภาพทีละภาพ และผู้ใช้ (ผู้พัฒนา) จะทำการคลิกเลือกจุดเป้าหมายบนเส้นกลางถนนที่ต้องการให้รถมุ่งหน้าไป หลักเกณฑ์ในการเลือกจุดคือการเลือกจุดกึ่งกลางของเลนที่มองเห็นในระยะข้างหน้าประมาณ 1/3 ของความสูงภาพจากด้านบน หรือบริเวณส่วนโค้งของเส้นที่รถควรจะตามไป

```
create_regression_labels.py > mouse_event_callback
1  import cv2
2  import os
3  import pandas as pd
4  import time
5
6  # --- Configuration for Annotation Script ---
7  SOURCE_DATASET_BASE_FOLDER = "dataset/car_control_combined" # << โฟลเดอร์หลักที่มีโฟลเดอร์ย่อย forward, left_turn ฯลฯ
8  OUTPUT_ANNOTATION_FILE = "regression_labels.csv" # << ชื่อไฟล์ CSV ที่จะบันทึก Label
9  TARGET_IMAGE_WIDTH_FOR_MODEL = 224 # << ขนาดความกว้างของภาพที่โมเดลจะใช้ (และใช้ในการ Normalize)
10 TARGET_IMAGE_HEIGHT_FOR_MODEL = 224 # << ขนาดความสูงของภาพที่โมเดลจะใช้ (และใช้ในการ Normalize)
11 # -----
12
13 annotations_data = []
14 current_image_display = None
15 clicked_point = None
16 current_file_info = {} # To store info about the current image
17
18 def mouse_event_callback(event, x, y, flags, param):
19     global clicked_point, current_image_display
20     if event == cv2.EVENT_LBUTTONDOWN:
21         clicked_point = (x, y)
22         # Draw a circle on the display image to confirm the click
23         img_copy = current_image_display.copy()
24         cv2.circle(img_copy, clicked_point, 5, (0, 255, 0), -1)
25         cv2.line(img_copy, (TARGET_IMAGE_WIDTH_FOR_MODEL // 2, TARGET_IMAGE_HEIGHT_FOR_MODEL), clicked_point, (0,0,255),1)
26         cv2.imshow("Annotate Image", img_copy)
27         print(f"Clicked at (display coords): {clicked_point}")
28
29 def run_image_annotation():
30     global clicked_point, current_image_display, current_file_info, annotations_data
31
32     # Load existing annotations if file exists to avoid re-annotating
```

(รูปที่ 3: ตัวอย่างโค้ดใน create_regression_labels.py)

หลังจากได้พิกัดจากการคลิก (x_pixel, y_pixel) บนภาพที่ถูก Resize ให้มีขนาดเท่ากับ Input ของโมเดล (224x224 pixels) แล้ว พิกัดดังกล่าวจะถูก Normalize ให้อยู่ในช่วง [-1, 1] โดยใช้สูตร: $x_{normalized} = (x_{pixel} / (224 - 1.0)) * 2.0 - 1.0$ $y_{normalized} = (y_{pixel} / (224 - 1.0)) * 2.0 - 1.0$

ข้อมูล Label ทั้งหมด (ชื่อไฟล์ภาพที่ถูกสร้างให้ unique และพิกัด x,y ที่ Normalize แล้ว) ถูกบันทึกลงในไฟล์ regression_labels.csv จากนั้น รูปภาพทั้งหมดจากโฟลเดอร์ย่อยเดิมได้ถูกคัดลอกมารวมกันในโฟลเดอร์

my_car_regression_data/images/ และเปลี่ยนชื่อไฟล์ให้ตรงกับคอลัมน์ image_filename ในไฟล์ CSV ดังกล่าว ซึ่งต่อมาได้เปลี่ยนชื่อเป็น labels.csv เพื่อใช้กับสคริปต์ Training

	A	B	C
1	image_filename	x_normalized	y_normalized
2	forward (1).jpg	-0.59641	0.0583
3	forward (10).jpg	-0.17489	0.21076
4	forward (100).jpg	0.04933	0.16592
5	forward (1000).jpg	-0.10314	0.14798
6	forward (1001).jpg	-0.10314	0.14798
7	forward (1002).jpg	-0.10314	0.14798
8	forward (1003).jpg	-0.10314	0.14798
9	forward (1004).jpg	-0.09417	0.19283
10	forward (1005).jpg	-0.09417	0.19283
11	forward (1006).jpg	-0.09417	0.19283
12	forward (1007).jpg	-0.09417	0.19283
13	forward (1008).jpg	-0.09417	0.19283
14	forward (1009).jpg	-0.09417	0.19283
15	forward (101).jpg	0.04036	0.2287
16	forward (1010).jpg	-0.11211	0.19283
17	forward (1011).jpg	-0.0852	0.19283
18	forward (1012).jpg	-0.11211	0.16592
19	forward (1013).jpg	-0.10314	0.16592
20	forward (1014).jpg	-0.09417	0.16592
21	forward (1015).jpg	-0.21076	0.13901

(รูปที่ 4: ตัวอย่าง output ที่ได้หลังจากบันทึกพิกัดแต่ละรูป)

ก่อนป้อนเข้าโมเดล รูปภาพแต่ละภาพจะถูก Resize ให้มีขนาด 224x224 pixels, แปลงเป็น RGB, แปลงเป็น PyTorch Tensor, และ Normalize ค่าสีของ Pixel โดยใช้ค่า Mean [0.485, 0.456, 0.406] และ Standard Deviation [0.229, 0.224, 0.225] ซึ่งเป็นค่ามาตรฐานสำหรับโมเดลที่ Pre-trained บน ImageNet

2.4 การแบ่งข้อมูล (Data Splitting)

ในการฝึกฝนโมเดลครั้งนี้ ไม่ได้มีการแบ่งชุดข้อมูล Validation Set แยกต่างหาก ข้อมูลทั้งหมดถูกใช้เป็น Training Set และการประเมินผลโมเดลเบื้องต้นจะทำการสังเกตพฤติกรรมของการขับเคลื่อนในสนามจำลอง Unity และการดูค่า Loss ระหว่างการเทรน

2.5 การเพิ่มข้อมูล (Data Augmentation)

เพื่อเพิ่มความหลากหลายให้กับชุดข้อมูลฝึกฝนและช่วยลดโอกาสการเกิด Overfitting ได้มีการใช้เทคนิค Data Augmentation พื้นฐานในสคริปต์ train_car_model.py ดังนี้

- **Random Horizontal Flip:** สุ่มพลิกภาพในแนวนอนด้วยความน่าจะเป็น 0.5 เพื่อให้โมเดลเรียนรู้จากภาพที่สะท้อนเสมือนวิ่งในเลนตรงข้าม (Label x จะต้องถูกกลับเครื่องหมายด้วยหากมีการ Flip ซึ่งในที่นี้ การ Flip จะทำก่อนการดึง Label ทำให้ Label ที่คู่กับภาพที่ Flip แล้วยังคงถูกต้องตามภาพนั้นๆ)
- **Color Jitter:** สุ่มปรับค่าความสว่าง (brightness), ความคมชัด (contrast), ความอิ่มตัวของสี (saturation), และเฉดสี (hue) เล็กน้อย เพื่อจำลองสภาพแสงที่แตกต่างกันในสนามแข่ง
- **Random Rotation:** สุ่มหมุนภาพเล็กน้อย (ไม่เกิน 15 องศา) เพื่อให้โมเดลทนทานต่อการเอียงของกล้องเล็กน้อย

```
train_car_model.py > ...
1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import Dataset, DataLoader
5  import torchvision.transforms as transforms
6  import torchvision.models as models
7  from PIL import Image
8  import pandas as pd
9  import os
10 import traceback # สำหรับแสดงข้อผิดพลาดโดยละเอียด
11
12 # --- Configuration ---
13 IMAGE_WIDTH = 224 # ความกว้างของรูปภาพที่ใช้เทรน (ควรตรงกับ ClientCarRacing.py)
14 IMAGE_HEIGHT = 224 # ความสูงของรูปภาพที่ใช้เทรน (ควรตรงกับ ClientCarRacing.py)
15 BATCH_SIZE = 32 # จำนวนตัวอย่างข้อมูลต่อ 1 batch (ปรับตาม VRAM/RAM)
16 NUM_EPOCHS = 60 # จำนวนรอบในการเทรนข้อมูลทั้งหมด (ปรับเพิ่ม/ลดตามผลลัพธ์)
17 LEARNING_RATE = 0.0005 # อัตราการเรียนรู้ของโมเดล (ค่าเริ่มต้นที่เหมาะสม)
18 DATASET_PATH = "my_car_regression_data"
19 CSV_FILENAME = "regression_labels.csv" # <<<< ชื่อไฟล์ CSV ที่มีข้อมูล Label (อยู่ใน DATASET_PATH)
20 IMAGE_FOLDER_NAME = "images" # <<<< ชื่อโฟลเดอร์ที่เก็บรูปภาพ (อยู่ใน DATASET_PATH)
21 MODEL_SAVE_FILENAME = "car_upgrade.pth" # ชื่อไฟล์สำหรับบันทึกโมเดลที่เทรนเสร็จ
22
23 # ตรวจสอบว่ามี GPU หรือไม่ และเลือก Device ที่จะใช้
24 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
25 print(f"Using device: {DEVICE}")
26
27 # --- AI Model Definition (ResNet18 for regression of x,y) ---
28 class RegressionResNet18(nn.Module):
29     def __init__(self, num_outputs=2): # Output 2 ค่าคือ x และ y
30         super(RegressionResNet18, self).__init__()
31         # โหลด Pre-trained ResNet18 จาก torchvision
32         self.resnet = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
```

(รูปที่ 5: ตัวอย่างโค้ดในไฟล์ train_car_model.py)

3. สถาปัตยกรรมโมเดล (Model Architecture)

โมเดลที่ใช้ในการควบคุมรถยนต์คือ Regression Model ที่มีพื้นฐานจากสถาปัตยกรรม ResNet18. ResNet18 เป็น Convolutional Neural Network (CNN) ที่มีชื่อเสียงและได้รับการพิสูจน์แล้วว่ามีประสิทธิภาพในการเรียนรู้ Feature จากรูปภาพ และได้รับการ Pre-trained บนชุดข้อมูล ImageNet ทำให้สามารถนำมา Fine-tune กับ Dataset เฉพาะทางได้ดี

โครงสร้างหลักของ ResNet18 ประกอบด้วย Convolutional Layers, Batch Normalization Layers, ReLU Activation Functions, และ Residual Connections สำหรับโปรเจกต์นี้ Fully Connected (FC) Layer สุดท้ายของ ResNet18 ที่เดิมทีใช้สำหรับ Classification 1000 classes บน ImageNet ได้ถูกปรับเปลี่ยนดังนี้: `self.resnet.fc = torch.nn.Sequential(torch.nn.Linear(num_fts, 2), torch.nn.Tanh())` โดย `num_fts` คือจำนวน Input Features ของ FC Layer เดิม (สำหรับ ResNet18 คือ 512). Layer ใหม่นี้เป็น `torch.nn.Sequential` ที่ประกอบด้วย

1. `torch.nn.Linear(num_fts, 2)`: Linear Layer ที่แปลง Input Features 512 features ให้ออกมาเป็น 2 Output Units ซึ่งแทนค่า `xnormalized` และ `ynormalized` ตามลำดับ
2. `torch.nn.Tanh()`: Tanh Activation Function ที่ทำหน้าที่บีบค่า Output ทั้งสองให้อยู่ในช่วง `[-1, 1]` ซึ่งสอดคล้องกับช่วงค่าของ Label ที่ได้ทำการ Normalize ไว้

4. การฝึกฝนโมเดล (Model Training)

4.1 สภาพแวดล้อมการฝึกฝน

- **Hardware:** CPU: Intel Core i5-11400H, GPU: NVIDIA GeForce RTX 3060
- **Software/Libraries:** [Python 3.9.15 , PyTorch 1.12.1, TorchVision 0.13.1, OpenCV-Python 4.6.0, Pandas, NumPy]

4.2 Hyperparameters

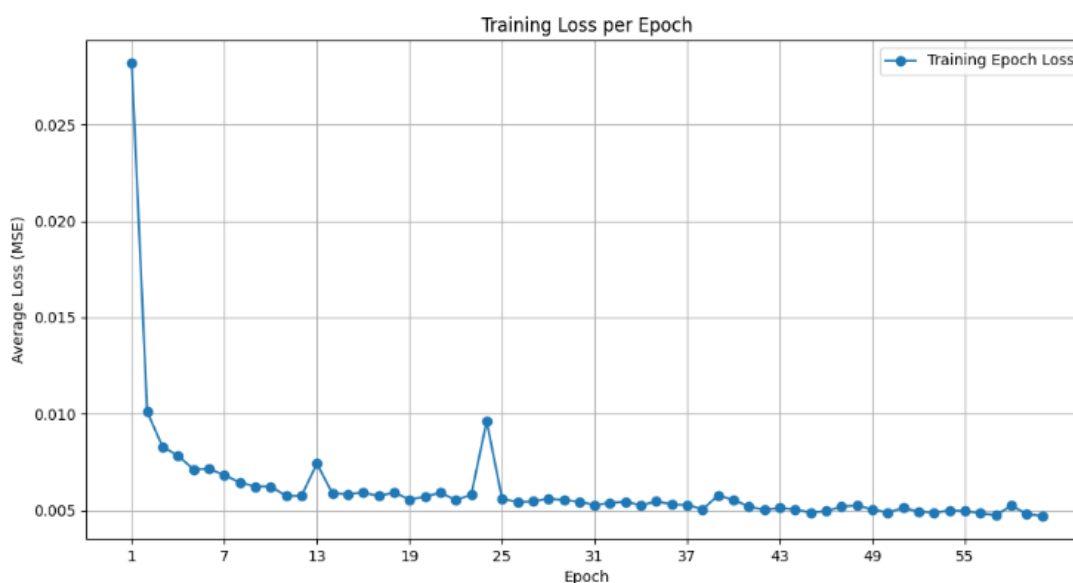
- **Loss Function:** Mean Squared Error (MSELoss) (`torch.nn.MSELoss()`) ถูกเลือกใช้เนื่องจากเป็น Loss Function มาตรฐานสำหรับงาน Regression ที่เป้าหมายคือการลดความคลาดเคลื่อนระหว่างค่าที่โมเดลทำนายกับค่าจริง (พิกัด x,y)
- **Optimizer:** Adam Optimizer (`torch.optim.Adam`)
 - **Learning Rate (LR):** 0.0005
 - **Betas:** 0.9
 - **Weight Decay:** 0
- **Batch Size:** 32
- **Number of Epochs:** 60

4.3 กระบวนการฝึกฝน

กระบวนการฝึกฝนโมเดลดำเนินการโดยใช้สคริปต์ `train_car_model.py` ในแต่ละ Epoch, ข้อมูลจาก Training Set จะถูกแบ่งออกเป็น Batch ตาม `BATCH_SIZE` ที่กำหนด สำหรับแต่ละ Batch

1. รูปภาพ Input และ Label (x,y) ถูกส่งไปยัง GPU
2. โมเดลทำการ Forward Pass เพื่อคำนวณ Output (predicted x,y)
3. คำนวณค่า Loss โดยใช้ MSELoss เพื่อเปรียบเทียบ Output ที่ได้กับ Label จริง
4. Optimizer ทำการ `zero_grad()` เพื่อล้าง Gradient จากรอบก่อนหน้า
5. ทำการ Backward Pass (`loss.backward()`) เพื่อคำนวณ Gradient ของ Loss เทียบกับ Parameters ของโมเดล
6. Optimizer ทำการอัปเดต Weights ของโมเดล (`optimizer.step()`) ค่า Loss เฉลี่ยของแต่ละ Epoch จะถูกบันทึกและแสดงผลเพื่อติดตามความคืบหน้าในการฝึกฝน

4.4 ผลลัพธ์การฝึกฝน (กราฟ Loss)



(รูปที่ 6: กราฟ Training Loss เทียบกับจำนวน Epochs)

กราฟนี้แสดงค่า Loss เฉลี่ย (Average Loss) ของชุดข้อมูลฝึกฝน (Training Set) ในแต่ละ Epoch ของการฝึกฝนโมเดล Regression. แกน X แทนจำนวน Epoch (ตั้งแต่ 1 ถึง 60) และแกน Y แทนค่า Loss เฉลี่ย ซึ่งในที่นี้คาดว่าเป็น Mean Squared Error (MSE) เนื่องจากเป็น Loss Function ที่ใช้สำหรับ Regression Model ที่เรากำลังพัฒนาขึ้น

จากกราฟ สามารถสังเกตเห็นแนวโน้มที่สำคัญดังนี้

1. การลดลงอย่างรวดเร็วในช่วงแรก (Epochs 1-7):

- ใน Epoch แรกสุด (Epoch 1) ค่า Loss เริ่มต้นอยู่ที่ประมาณ 0.028 ซึ่งค่อนข้างสูง
- หลังจากนั้น ค่า Loss ลดลงอย่างรวดเร็วและชันมากในช่วงประมาณ 1-7 Epochs แรก โดยลดลงมาอยู่ที่ประมาณ 0.008 ใน Epoch ที่ 7
- Điều này cho thấy mô hình học được các đặc trưng vàรูปแบบพื้นฐานจากข้อมูลได้อย่างรวดเร็วในช่วงเริ่มต้นของการฝึกฝน

2. การ Converge และความผันผวน (Epochs 7-60):

- **แนวโน้มโดยรวม:** หลังจาก Epoch ที่ 7 ค่า Loss โดยรวมยังคงมีแนวโน้มลดลง แต่ในอัตราที่ช้าลงมาก และเริ่มมีการแกว่งตัว (Fluctuation) หรือ "กระโดด" ของค่า Loss ในบางช่วง
- **จุดที่น่าสนใจ:**
 - **Epoch 13-14:** มีการเพิ่มขึ้นเล็กน้อยของ Loss ก่อนที่จะลดลงอีกครั้ง
 - **Epoch 23-24:** มีการ "กระโดด" ขึ้นของค่า Loss อย่างเห็นได้ชัดไปที่ประมาณ 0.0097 ก่อนที่จะลดลงมาอยู่ในระดับเดิม
 - **หลังจาก Epoch 25:** ค่า Loss โดยรวมค่อนข้างคงที่และแกว่งตัวอยู่ในช่วงแคบๆ ระหว่างประมาณ 0.0045 ถึง 0.006 แม้จะมีการแกว่งตัวขึ้นลงเล็กน้อย แต่ไม่ได้มีแนวโน้มลดลงอย่างชัดเจนเหมือนช่วงแรก
- **การตีความ:**
 - การที่ Loss ลดลงช้าลงและเริ่มแกว่งตัว บ่งชี้ว่าโมเดลอาจจะเริ่มเข้าใกล้จุด Optimal (หรือ Local Minimum) สำหรับชุดข้อมูลฝึกฝน
 - การ "กระโดด" ของ Loss (Epoch 24) อาจเกิดจากหลายปัจจัย เช่น Learning Rate ที่อาจจะยังสูงไปสำหรับช่วงนั้น ทำให้โมเดล "กระโดด" ข้ามจุด Optimal, หรือลักษณะของข้อมูลในบาง Batch ที่ทำให้ Gradient เปลี่ยนทิศทางอย่างกะทันหัน
 - การที่ Loss ค่อนข้างนิ่งในช่วงท้าย (Epoch 40-60) แต่ยังคงมีการแกว่งตัวเล็กน้อย อาจหมายความว่าโมเดลได้เรียนรู้จาก Training Data ไปมากแล้ว และการปรับปรุงเพิ่มเติมอาจจะต้องใช้เทคนิคอื่น เช่น การเพิ่มข้อมูลที่มีความหลากหลายมากขึ้น

3. ค่า Loss สุดท้าย:

- ใน Epoch สุดท้าย (ประมาณ Epoch 60) ค่า Loss เฉลี่ยอยู่ที่ประมาณ 0.0048 - 0.005
- ค่านี้แสดงถึงระดับความคลาดเคลื่อนเฉลี่ยกำลังสอง (Mean Squared Error) ระหว่างค่าพิกัด (x,y) ที่โมเดลทำนายกับค่าพิกัดจริงบนชุดข้อมูลฝึกฝน ค่าที่น้อยลงบ่งบอกว่าโมเดลทำนายได้ใกล้เคียงกับค่าจริงมากขึ้นบนชุดข้อมูลที่ใช้เทรน

สรุปจากกราฟ

โมเดลแสดงให้เห็นถึงความสามารถในการเรียนรู้จากข้อมูล Training Set โดยมีค่า Loss ลดลงอย่างมากในช่วงแรก และเริ่ม Converge ในช่วงหลัง การแกว่งตัวของ Loss ในบางช่วงอาจบ่งชี้ถึงโอกาสในการปรับปรุง Hyperparameters หรือการเพิ่มเทคนิค Regularization เพื่อให้การ Converge ราบรื่นขึ้นและป้องกัน Overfitting โดยรวมแล้ว กราฟนี้แสดงให้เห็นว่ากระบวนการฝึกฝนดำเนินไปในทิศทางที่ถูกต้อง และโมเดลได้เรียนรู้ที่จะลดความผิดพลาดในการทำนายบนชุดข้อมูลที่ใช้ฝึกฝน

5. การทดสอบและประเมินผลโมเดล (Model Testing and Evaluation)

5.1 วิธีการทดสอบ

การประเมินผลโมเดลหลักทำผ่านการทดสอบเชิงคุณภาพโดยนำโมเดลที่ฝึกฝนแล้วไปใช้งานจริงกับรถยนต์จำลองในสนามแข่งของโปรแกรม Unity ผ่านสคริปต์ ClientCarRacing.py การทดสอบนี้เน้นการสังเกตพฤติกรรมการขับขี่ของรถยนต์ในสถานการณ์ต่างๆ เช่น การวิ่งในทางตรง, การเข้าโค้งซ้ายและขวา, และการรักษาดำแหน่งกลางเลน

5.2 ผลการทดสอบเชิงคุณภาพ (พฤติกรรมรถใน Unity)

- การขับในทางตรง: โมเดลสามารถควบคุมรถให้วิ่งตรงในเลนได้ค่อนข้างดี มีการแกว่งเล็กน้อยเมื่อ `pred_x_norm` มีค่าใกล้ศูนย์ แต่โดยรวมสามารถรักษาสถานะได้
- การเข้าโค้ง: ในการเข้าโค้งซ้ายและขวา โมเดลสามารถทำนายทิศทางได้ถูกต้อง ทำให้รถเลี้ยวตามโค้งได้
- การทำงานของ Confidence: การเพิ่ม Logic `CONFIDENCE_THRESHOLD_X_NORM` ช่วยลดการส่ายของรถในทางตรงได้ เมื่อ AI ไม่ได้ทำนายค่า `x_norm` ที่ชัดเจนว่าจะไปซ้ายหรือขวามากนัก รถจะพยายามวิ่งตรงไป ซึ่งช่วยให้การขับดีขึ้น
- ข้อจำกัดที่พบ: โมเดลยังคงมีปัญหาคือในขณะเลี้ยวรถจะไม่ทำการเคลื่อนที่ระหว่างเลี้ยว แต่จะแค่หันหัวของรถให้ตรงกับเส้นในเลนเท่านั้น



(รูปที่ 7: การเคลื่อนที่ของรถยนต์ใน unity)

6. การนำโมเดลไปใช้ใน ClientCarRacing.py

6.1 การโหลดโมเดล

โมเดลที่ฝึกฝนแล้ว (car_upgrade.pth) ถูกโหลดเข้ามาในสคริปต์ ClientCarRacing.py โดยเริ่มต้นจากการสร้าง Instance ของ RegressionResNet18 จากนั้น torch.load(MODEL_PATH, map_location=device) จะถูกใช้เพื่อโหลด state_dict ที่บันทึกไว้

6.2 การประมวลผลภาพ Input

ภาพที่ได้รับจาก Unity ผ่าน UDP Stream ในรูปแบบ BGR จะถูกประมวลผลโดยฟังก์ชัน `preprocess_image_for_model`. ขั้นตอนประกอบด้วย

1. Resize ภาพให้มีขนาด `IMAGE_WIDTH (224) x IMAGE_HEIGHT (224) pixels`
2. แปลงสีภาพจาก BGR เป็น RGB
3. แปลงภาพเป็น PIL Image format
4. ใช้ `torchvision.transforms` เพื่อแปลงเป็น PyTorch Tensor และ Normalize ค่า Pixel โดยใช้ Mean `[0.485, 0.456, 0.406]` และ Standard Deviation `[0.229, 0.224, 0.225]`
5. เพิ่ม Dimension ของ Batch (`unsqueeze(0)`) และส่ง Tensor ไปยัง GPU

6.3 การตีความ Output และการแปลงเป็นคำสั่งควบคุม

ภายในฟังก์ชัน `ai_control_loop`

1. โมเดล AI (`ai_model`) รับ Input Tensor และให้ Output เป็น `prediction_xy_normalized` ซึ่งมี 2 ค่า คือ `pred_x_norm` และ `pred_y_norm` (ค่า Normalized ในช่วง `[-1, 1]`)
2. `pred_x_norm` ถูกนำมา Denormalize เป็น `pred_x_pixel` (พิกัดบนแกน x ของภาพ) โดยใช้สูตร:
$$(\text{pred_x_norm} + 1.0) * 0.5 * (\text{IMAGE_WIDTH} - 1)$$
3. `error_x` คำนวณจาก `pred_x_pixel - CENTER_X_IMAGE`
4. `steering_angle` คำนวณจาก `error_x * STEERING_GAIN` (`STEERING_GAIN = 1`)
5. Logic การตัดสินใจส่งคำสั่ง
 - มีการตรวจสอบ "ความมั่นใจ" โดยใช้ `CONFIDENCE_THRESHOLD_X_NORM = 0.6` หาก `abs(pred_x_norm)` น้อยกว่าค่านี้ และ `abs(steering_angle)` ไม่ได้สูงเกิน `steering_command_threshold + 10.0` รถจะถูกสั่งให้ "Move Forward" ด้วยความเร็ว `MAX_SPEED = 36`

- คำสั่งจะถูกส่งไปยัง ServerLanUDP.py ทุกๆ command_send_interval (0.05 วินาที) พร้อม duration ที่เหมาะสม



(รูปที่ 8: ตัวอย่างการทำงานของ AI Controlled Car Vision)

7. การประยุกต์ใช้ Generative AI และ Prompt Engineering

7.1 วัตถุประสงค์ในการใช้ Generative AI

ในโปรเจกต์นี้ Generative AI (Gemini) ถูกใช้เป็นเครื่องมือช่วยในหลายส่วนของกระบวนการพัฒนา ได้แก่

- **การสร้างและปรับปรุง Logic การควบคุม:** การขอแนวคิดในการเพิ่ม Logic "Confidence Score" และการทำให้รถเคลื่อนที่ไปข้างหน้าขณะเลี้ยว

- การสร้างโครงร่างโค้ดเบื้องต้น: สำหรับสคริปต์บางส่วน เช่น สคริปต์ Annotation และ สคริปต์ Training
- การทำความเข้าใจแนวคิด: ขอคำอธิบายเกี่ยวกับ Parameter ต่างๆ

7.2 ตัวอย่าง Prompt ที่ใช้และเหตุผลในการออกแบบ

- Prompt 1
 - Prompt: "จากโค้ด ClientCarRacing.py ฉันอยากให้ระหว่างเลียวซ้าย หรือ เลี้ยวขวา รอยยังคง forward ไปพร้อมกับเลียวอยู่ด้วย ด้วยความเร็วที่ลดลง"
 - เหตุผลในการออกแบบ Prompt: Prompt นี้มีความชัดเจนและตรงประเด็น ระบุไฟล์ที่ต้องการแก้ไข (ClientCarRacing.py), พฤติกรรมที่ต้องการ (forward ไปพร้อมกับเลียว), และเงื่อนไขเพิ่มเติม (ด้วยความเร็วที่ลดลง) เพื่อให้ AI เข้าใจความต้องการและสามารถให้คำแนะนำหรือโค้ดที่เกี่ยวข้องได้
- Prompt 2
 - Prompt: "ฉันสามารถใส่ค่าความมั่นใจของ AI ได้ไหม เช่น หากไม่มั่นใจว่าเลียวซ้ายหรือขวาเกิน 0.6 ให้ตรงไป"
 - เหตุผลในการออกแบบ Prompt: Prompt นี้เริ่มจากการตั้งคำถามถึงความเป็นไปได้ ("ฉันสามารถใส่ค่าความมั่นใจได้ไหม") และตามด้วยตัวอย่างที่เป็นรูปธรรมของเงื่อนไขที่ต้องการ ("หากไม่มั่นใจว่าเลียวซ้ายหรือขวาเกิน 0.6 ให้ตรงไป") ซึ่งช่วยให้ AI เข้าใจ Context และเสนอวิธีการ Implement ที่เหมาะสมได้
- Prompt 3
 - Prompt: ฉันขอโค้ดสำหรับเทรนโมเดลเพื่อควบคุมรถให้เคลื่อนที่ตามเส้นกลางถนน โดยอิงจาก dataset สำหรับการเทรน
 - เหตุผลในการออกแบบ Prompt: Prompt นี้ระบุเป้าหมายหลัก ("เทรนโมเดลเพื่อควบคุมรถให้เคลื่อนที่ตามเส้นกลางถนน") และข้อมูล Input ที่มี ("dataset สำหรับการ

การเทรน") ทำให้ AI สามารถสร้างสคริปต์ Training ที่เกี่ยวข้องกับงาน Regression ได้

7.3 ผลลัพธ์ที่ได้จาก Generative AI

- **จาก Prompt 1:** AI ได้ให้แนวทางการแก้ไขโค้ดใน `ai_control_loop` โดยการเพิ่มตัวแปร `TURNING_SPEED` และปรับการเรียกใช้ฟังก์ชัน `send_command()` ให้มีการส่งค่า `speed` ขณะเลี้ยว
- **จาก Prompt 2:** AI ได้อธิบายวิธีการตีความ "ความมั่นใจ" จาก Output ของ Regression Model และเสนอแนวทางการเพิ่ม Logic โดยใช้ `CONFIDENCE_THRESHOLD_X_NORM` เพื่อเปรียบเทียบกับ `pred_x_norm`
- **จาก Prompt 3:** AI ได้ให้โค้ดตัวอย่างสำหรับสคริปต์ `train_car_model.py` ซึ่งประกอบด้วย Model Definition, Dataset Class, Training Loop, Loss Function, และ Optimizer ที่เหมาะสมสำหรับงาน Regression

7.4 การวิเคราะห์และการพัฒนาต่อยอดจากผลลัพธ์ของ AI

- **สำหรับ Prompt 1 และ 2:** นำโค้ดและแนวคิดที่ AI แนะนำมาปรับใช้ใน `ai_control_loop` ของ `ClientCarRacing.py` ทดลองปรับค่า `TURNING_SPEED` และ `CONFIDENCE_THRESHOLD_X_NORM` รวมถึงเงื่อนไขเพิ่มเติมเพื่อให้การควบคุมรถมีความสมดุลระหว่างการตอบสนองและความนิ่ง ไม่ได้คัดลอกโค้ดมาทั้งหมด แต่ทำความเข้าใจหลักการแล้วปรับให้เข้ากับ Parameter อื่นๆ ของโปรเจกต์
- **สำหรับ Prompt 3:** ใช้โค้ด `train_car_model.py` ที่ AI ให้มาเป็นฐานในการสร้างสคริปต์ Training ของตนเอง โดยมีการปรับแก้ Path, Hyperparameters (Learning Rate, Batch Size, Epochs) และส่วนการจัดการ Dataset ให้สอดคล้องกับไฟล์ `labels.csv` ที่สร้างขึ้นจากการ Annotation

7.5 การประเมินผลการใช้ Generative AI

- **ประโยชน์:** Generative AI (Gemini) มีประโยชน์อย่างมากในการช่วยเร่งกระบวนการพัฒนา โดยเฉพาะการแก้ไขปัญหาทางเทคนิคที่ซับซ้อน เช่น การ Debug Error ของ PyTorch และการ

ให้แนวคิดในการปรับปรุง Logic การควบคุมรถ นอกจากนี้ยังช่วยในการร่างโครงสร้างโค้ด เบื้องต้นและโครงร่างรายงาน ทำให้ประหยัดเวลาและสามารถเริ่มต้นได้เร็วขึ้น

- **ข้อดี:** สามารถให้คำแนะนำได้หลากหลายและรวดเร็ว, ช่วยอธิบายแนวคิดที่ซับซ้อนให้เข้าใจง่ายขึ้น, และเป็นเครื่องมือที่ดีสำหรับการเรียนรู้และทดลองแนวทางใหม่ๆ
- **ข้อจำกัด:** บางครั้งโค้ดที่ AI ให้มาอาจจะไม่สมบูรณ์ 100% หรือต้องมีการปรับแก้เพื่อให้เข้ากับ Context ของโปรเจกต์ทั้งหมด ผู้ใช้ยังคงต้องมีความเข้าใจพื้นฐานในเรื่องนั้นๆ เพื่อสามารถวิเคราะห์และปรับปรุงผลลัพธ์ที่ได้ การเขียน Prompt ที่เฉพาะเจาะจงและให้ข้อมูลเพียงพอเป็นสิ่งสำคัญเพื่อให้ได้ผลลัพธ์ที่มีประโยชน์สูงสุด
- **สิ่งที่ได้เรียนรู้:** ได้เรียนรู้ทักษะในการตั้งคำถาม (Prompt Engineering) ที่มีประสิทธิภาพกับ AI การให้ Context ที่ชัดเจน การแบ่งปัญหาใหญ่เป็นปัญหาย่อยๆ และการทำความเข้าใจในผลลัพธ์ที่ AI สร้างให้เพื่อนำมาพัฒนาต่อยอดเป็นงานของตนเอง การใช้ AI ไม่ได้มาแทนที่ความเข้าใจและการลงมือทำของผู้พัฒนา แต่เป็นเครื่องมือที่ทรงพลังในการเสริมสร้างกระบวนการเหล่านั้น

8. ปัญหาและอุปสรรค (ส่วนที่ไม่ได้ใช้ Generative AI ช่วย)

- **การ Annotation ข้อมูล:** การสร้าง Label (x,y) สำหรับ Regression Model เป็นกระบวนการที่ใช้เวลาและความละเอียดรอบคอบสูง การคลิกจุดเป้าหมายให้มีความสม่ำเสมอสำหรับรูปภาพจำนวนมากเป็นเรื่องท้าทาย
- **การปรับจูน Hyperparameters:** การหาค่า STEERING_GAIN, MAX_SPEED, TURNING_SPEED, CONFIDENCE_THRESHOLD_X_NORM, และ steering_command_threshold ที่เหมาะสมที่สุดใน ClientCarRacing.py เพื่อให้รถมีการตอบสนองที่ดีในสนามแข่งจริง (Unity) ต้องใช้การทดลองและปรับแก้หลายครั้ง
- **ความหลากหลายของ Dataset:** แม้จะมีการ Augmentation แต่ Dataset ที่ใช้อาจจะยังไม่ครอบคลุมทุกสถานการณ์ในสนามแข่งที่ซับซ้อน ทำให้โมเดลอาจจะยังทำงานได้ไม่ดีในบางจุดที่ไม่เคยเห็น

9. สรุปและข้อเสนอแนะ (Conclusion and Future Work)

โปรเจกต์นี้ประสบความสำเร็จในการพัฒนาระบบควบคุมรถยนต์จำลองโดยใช้ Regression Model (ResNet18) เพื่อทำนายพิกัดเป้าหมายจากภาพ และนำมาควบคุมการเคลื่อนที่ของรถยนต์ในโปรแกรม Unity ได้ มีการเพิ่ม Logic การตรวจสอบความมั่นใจของ AI และการปรับให้รถเคลื่อนที่ไปข้างหน้าขณะเลี้ยว ซึ่งช่วยให้การขับที่มีความสมจริงและนิ่งมากขึ้น การประยุกต์ใช้ Generative AI (Gemini) ผ่าน Prompt Engineering มีบทบาทสำคัญในการช่วยแก้ไขปัญหาค้าง, สร้างแนวคิด, และเร่งกระบวนการพัฒนาในหลายส่วน

ข้อเสนอแนะสำหรับการพัฒนาต่อไปในอนาคต

- ปรับปรุงกระบวนการ Annotation ข้อมูลให้มีประสิทธิภาพและแม่นยำยิ่งขึ้น หรือพิจารณาเทคนิค Semi-supervised Learning
- ทดลองสถาปัตยกรรมโมเดลอื่น หรือเพิ่มความซับซ้อนของโมเดลปัจจุบัน
- เพิ่ม Feature ต่างๆ
- พัฒนาระบบให้สามารถเรียนรู้และปรับตัว (Adaptive Control) กับสภาพสนามที่เปลี่ยนแปลงได้
- สำรวจเทคนิค Reinforcement Learning สำหรับการควบคุมรถยนต์